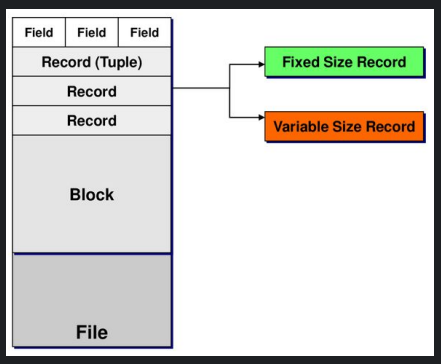
॥ श्री गणेशाय नम: ॥

File Organization and Indexing



A db is a collection of files (tables). A file is a collection of blocks (pages). A block is a collection of records. We move contents from the disk to the memory in a block-by-block manner.

Records in db tables are of 2 types:

a. fixed-length records:

E.g. consider a table:

```
CREATE TABLE R (
    A CHAR(100),
    B CHAR(100),
    C CHAR(200)
);
```

The size of a record would be 400B.

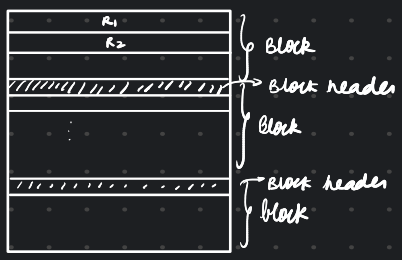
b. variable-length records:

E.g.

```
CREATE TABLE S (
    D CHAR(100),
    E CHAR(100),
    F TEXT              – variable length
);
```

The size of the TEXT is not known.

Blocks:

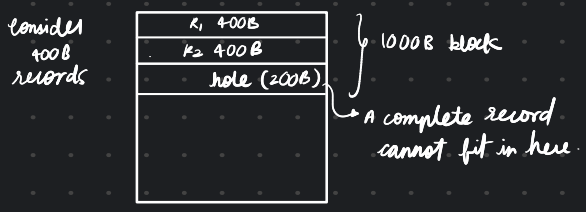


A block header contains the offset table to access records.

Record organization in DB files

Unspanned

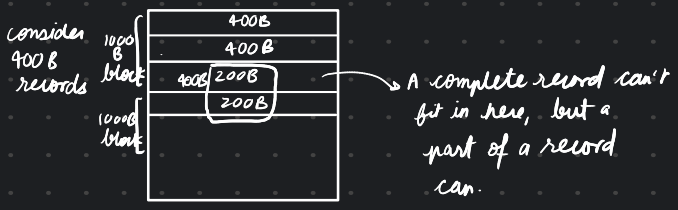
- a complete record must be stored in a single block.



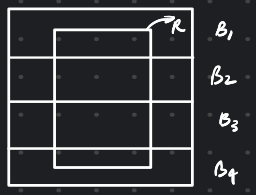
- Fixed-length records use unspanned organization.

Spanned

- a record can span multiple blocks.



- If a record spans multiple blocks, to get a single record, more disk accesses are required.



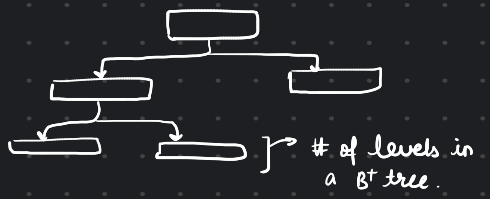
- Variable-length records use spanned organization.

Block factor: The maximum # of records per block.

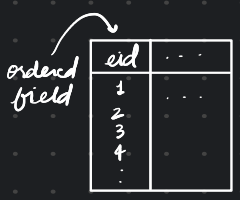
If
block size = B bytes,
header size = H bytes,
record size = R bytes,

$$BF = \left\lfloor \frac{B-H}{R} \right\rfloor$$

I/O Cost: The # of disk blocks (DB file blocks) required to be transferred from the disk to the main memory to access a required record.



A DBMS generally organizes a table in a physical order based on a particular field. That field is called an ordered field of the file.



Consider an example:



I/O cost to access a record from a DB table w/o an index:

a. using an ordered field

```
SELECT * FROM Emp
WHERE eid = n;
```

→ we could use binary search.

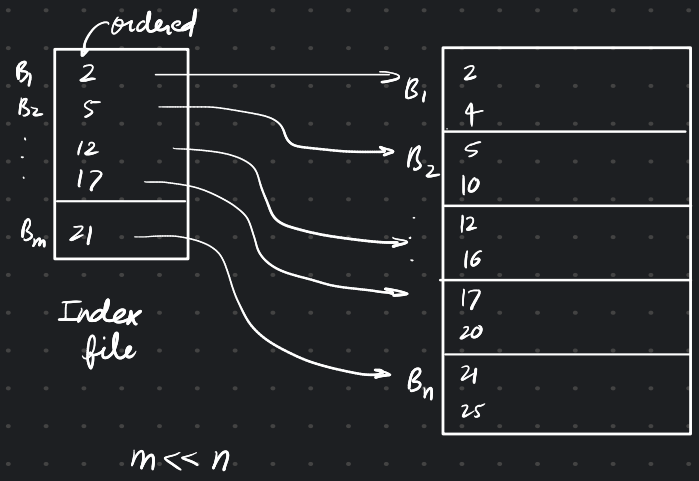
Worst case I/O cost = $\lceil \log_2 n \rceil$ blocks

b. using an unordered field

```
SELECT * FROM Emp
WHERE passport = n
```

Worst case I/O cost = n blocks

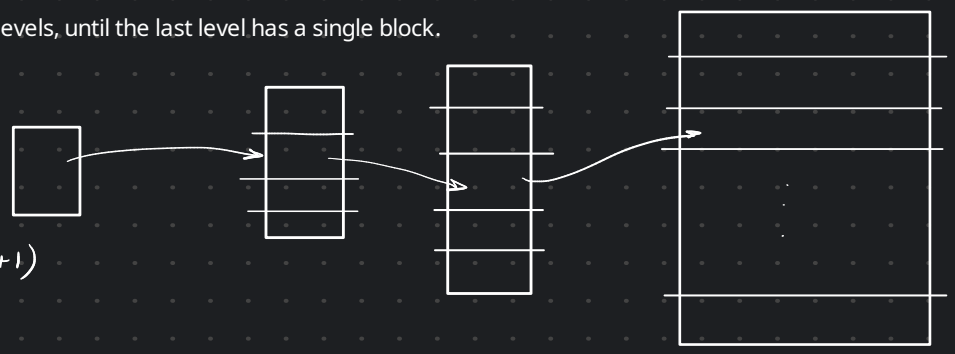
I/O cost to access a record from a DB file using indexes:



$m << n$

I/O cost = $\lceil \log_2 m \rceil$ + 1 block

To reduce the I/O cost further, we use multi-level indexes:
Multi-level indexing:
We have indexes to indexes at levels, until the last level has a single block.



Each entry of an index file has 2 fields:

⟨ search key, pointer ⟩

K bytes P bytes

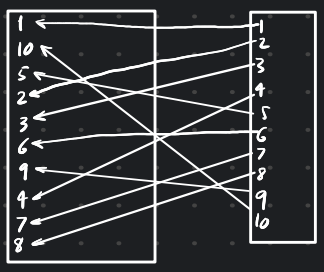
Block size = B bytes

index records in 1 block = BF = $\left\lfloor \frac{B-H}{K+P} \right\rfloor$ entries/block.

Types of indexes

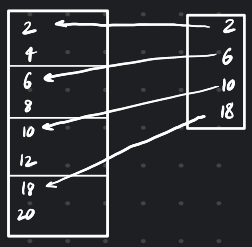
Dense indexes:
- These have a greater # of index entries.
- For each record in the DB file, there's an entry in the index file.
- These can be built on ordered or unordered fields.


Dense indexing on ordered key



Dense indexing on unordered key

Sparse indexes:
- These have a lesser # of index entries.
- For a set of records in the DB file, there's an entry in the index file.
- These can be built on ordered fields only.



If a sparse index is built on a candidate key, then # of index file entries = # of disk blocks

Q. There are 50,000 records in a DB table. A block occupies 1024 B. A search key occupies 12 B. A record occupies 100 B. A pointer occupies 6 B.
If a dense index is built, what is the
a. # of I level dense index blocks
b. I/O cost using I level indexes?

NOTE: Only the 1st level in multi-level indexes is dense.

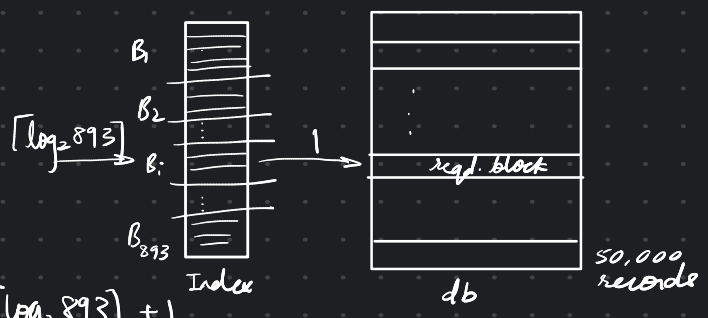
→ # of records = 50,000.
B = 1024 B.
K = 12B, P = 6B; K+P = 18 B
R = 100B
BF = $\left\lfloor \frac{1024}{100} \right\rfloor$ = 10
of index blocks/file = $\left\lceil \frac{B-H}{K+P} \right\rceil$ = $\left\lceil \frac{1024-0}{18} \right\rceil$ = 56

⇒ each block has 56 entries.

∴ 1 entry requires $\frac{1}{56}$ blocks.

50,000 entries require $\frac{50,000}{56}$ blocks.

= 893 blocks.



∴ I/o cost using single-level indexing using $\lceil \log_2 893 \rceil + 1$

$= \lceil 9.802 \rceil + 1 = 10$

Q. # of records = 50,000, block = 1024 B, search key = 12 B, record = 100 B, pointer size = 6 B.
If sparse indexing is used, what is the
a. # of 1st level sparse blocks
b. i/o cost using 1st level indexing
?

→

In sparse indexes, we'll have 1 entry / block.

BF of db table $= \left\lfloor \frac{B-H}{R} \right\rfloor = \left\lfloor \frac{1024}{100} \right\rfloor$ = 10 records / block.

∴ 1 record needs $\frac{1}{10}^{th}$ block.

50,000 records need $\frac{50,000}{10}$ = 5,000 blocks.

of index records = 5,000.

BF of index file $= \left\lfloor \frac{B-H}{K+P} \right\rfloor = \left\lfloor \frac{1024}{18} \right\rfloor = 56$ (entries per block)

∴ 5,000 entries will take $\lceil 5000 \times \frac{1}{56} \rceil$ blocks

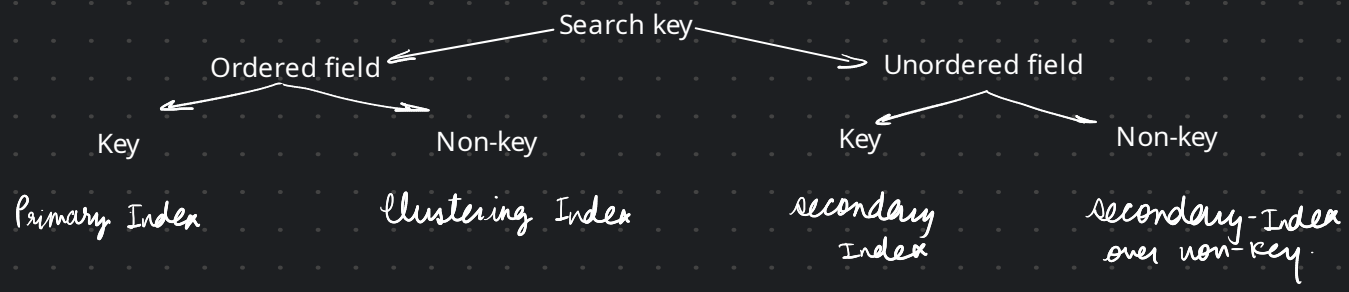
$= \lceil 89.28 \rceil$ = 90 blocks.

I/o cost

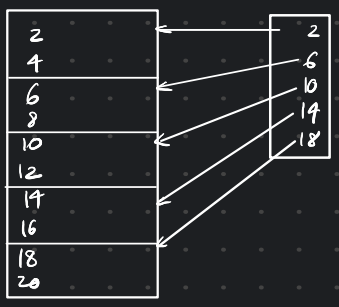
$= \lceil \log_2 90 \rceil + 1$

$= \lceil 6.49 \rceil + 1 = 7$

Search key:
It is the field used for indexing.
A search key can be the key of a DB table, or some non-key element.

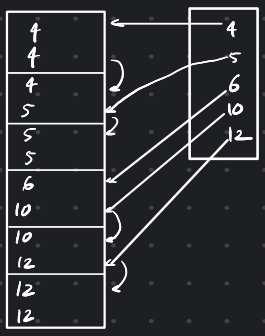


Primary index:
- ordered field (sorted) and key (non-repetition)

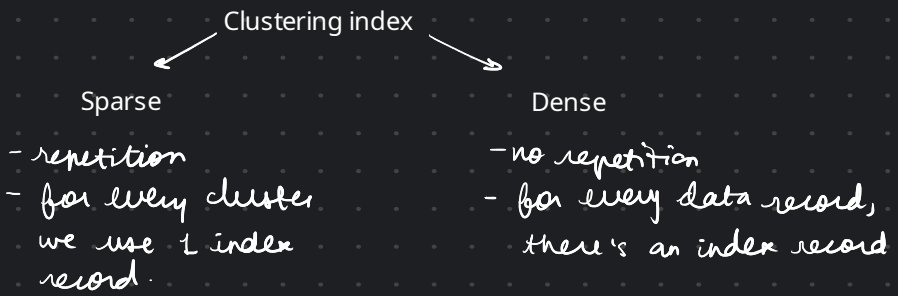


- Primary indexes can be both sparse and dense.
- I/O cost of PIs with multilevel indexing = k+1 (k=# of levels).
- For a DB table, there can be atmost 1 primary index, because the primary index is on sorted fields and we can have only 1 field on which the table can be sorted.

Clustering index:
- ordered field (sorted) and non-key (repetition)

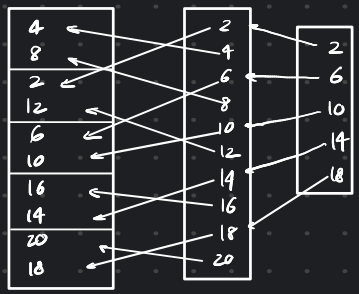


- I/O cost of CIs with multilevel indexing = k+1/more blocks (k=# of levels).
- A clustered index is mostly sparse.
- Atmost 1 clustering index is possible (as it is on a sorted fields).



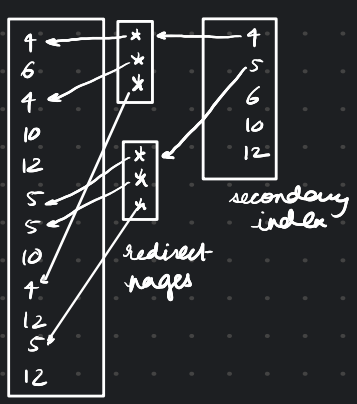
Either primary index or clustering index, only one can be present at a time.

Secondary index:
- uordered field (unsorted) and key (no repetition)



- A secondary index on unordered key fields is always dense.
- I/O cost = k+1 (k=# of levels of indexing).
- Many secondary indexes are possible (as they're not on ordered fields).

Secondary index over non-key fields:
- uordered field (unsorted) and non-key (repetition)
- A secondary index on unordered non-key fields is mostly sparse.



Dynamic multi-level indexes

Search tree

A search tree is a data structure that organizes keys.



Binary Search tree

A binary search tree is optimized for searching.



Worst-case time complexity for searching = O(n)



The tree may grow skewed.

We need a balanced search tree with worst case time complexity = O(logn) rather than O(n).

Balanced search trees

Max height of search tree should not exceed O(logn) for n distinct keys.
Worst case search cost doesn't exceed O(logn).

Balanced search trees

AVL trees / Red/black trees

- height-restricted BST
- degree = 2



B trees / B⁺ trees

- height-restricted search trees
- degree = p



- a node can have many keys and many pointers

E.g. of B-tree:

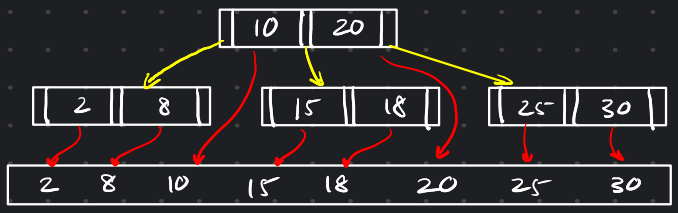


B-tree

Order p: It is the maximum # of possible block pointers that can be present in a B-tree node.

Node structure





• → block pointer
• → record pointer



Every internal node (except the root) must have atleast $\lceil \frac{p}{2} \rceil$ block pointers with $\lceil \frac{p}{2} \rceil - 1$ keys and atmost p block pointers and $(p-1)$ keys.

The root node can have atleast 2 block pointers with 1 key and atmost p block pointers with (p-1) keys.
Every leaf node must be at the same level.

E.g. order $p = 15$ (max $BPs = 15$).

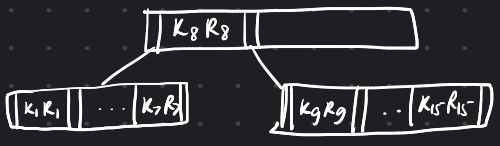


The node is full.
Say we want to add a $K_{15} R_{15}$.

We'll write the keys in a sorted order.

$K_1 < K_2 < \cdots < K_8 < \cdots < K_{15}$

$K_8 R_8$ will be pushed to make a new tree.



In B-trees, all nodes (except the root) are 50% full.

According to Knuth's definition, a B-tree of order *m* is a tree which satisfies the following properties:[7]

1. Every node has at most *m* children.
2. Every node, except for the root and the leaves, has at least ⌈*m*/2⌉ children.
3. The root node has at least two children unless it is a leaf.
4. All leaves appear on the same level.
5. A non-leaf node with *k* children contains *k*-1 keys.

Constructing a B tree

Order = 5
(max BPs = 5, max keys = 4)

Keys: 40, 60, 20, 50, 70, 90, 80, 75, 10, 15, 25, 55, 65, 68, 85, 98, 95.



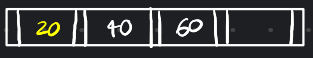
40



60



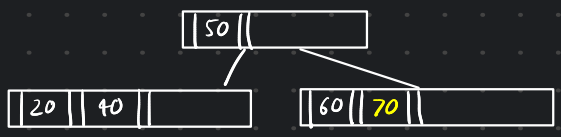
20



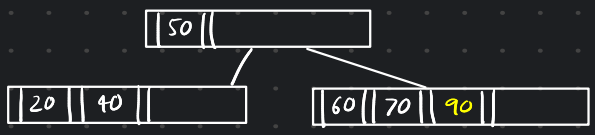
50



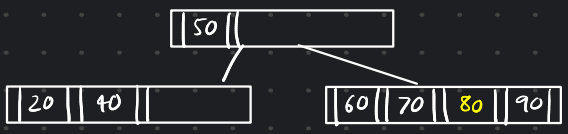
70 [20 40 50 60 70]



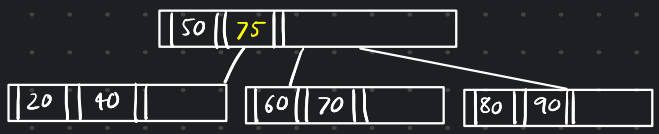
90



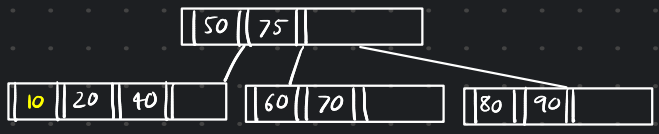
80



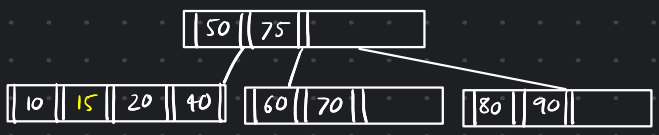
75 [60, 70, 75, 80, 90]



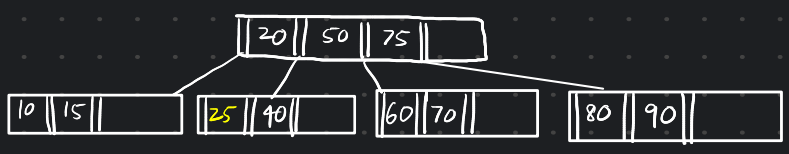
10



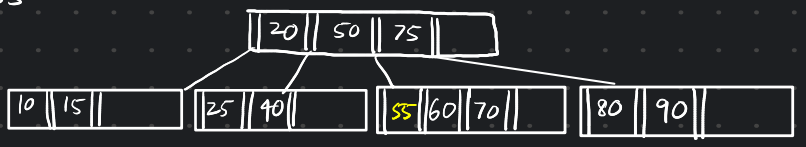
15



25 [10 15 20 25 90]



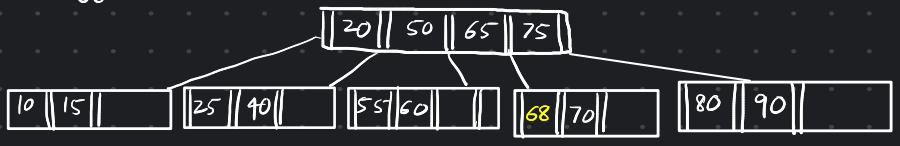
55



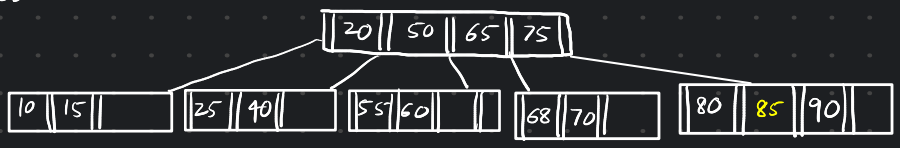
65



68



85



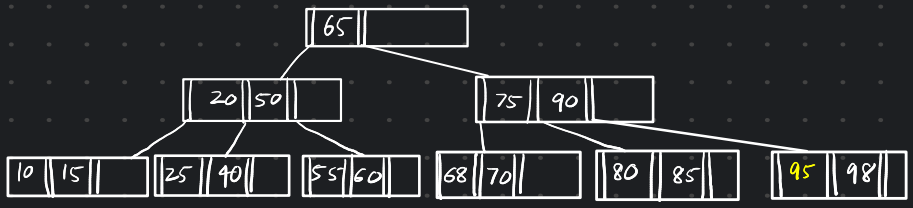
98



95

[20 50 65 75 90]

[80, 85, 90, 95, 98]





DB table: [40 60 20 50 70 90 80 75 10 15 25 55 65 68 85 98 95]

If the indexing is done on ordered fields, we get a sparse index. We need only 1 key per block.
If the indexing is done on unordered fields, we get a dense index. We need 1 key per db record.

B trees are suitable for random access of one record.

E.g. SELECT * FROM R
WHERE A = 40 ; I/O cost = K+1

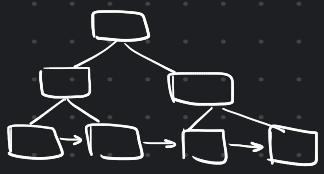
B trees are NOT suitable for sequential access of records (range queries).

E.g. SELECT * FROM R
WHERE A ≥ 10 AND A ≤ 100 ;

I/O cost : (K+1) for A=10,
(K+1) for A=11,
:
(K+1) for A=100

B+ trees are suitable for random access as well as sequential access of records.

B^+ trees

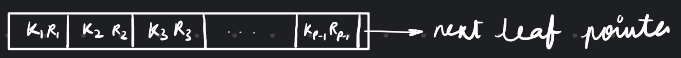


Order p: maximum possible # of pointers that can be stored in a B^+ tree node.

Node structure

Leaf node of a B^+ tree: set of (key, record pointer) pairs + 1 block pointer pointing to the next leaf.

Total # of pointers = p-1 + 1 = p.



Internal node of a B^+ tree: set of records with block pointers to child nodes.

Total # of pointers = p

Total # of keys = (p-1)



NO record pointer here



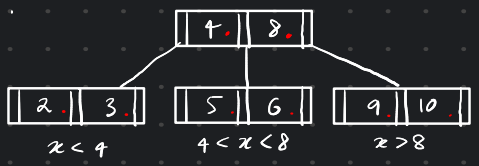
p BPs

(p-1) keys

In most cases, p is the maximum # of block pointers. But this may change.

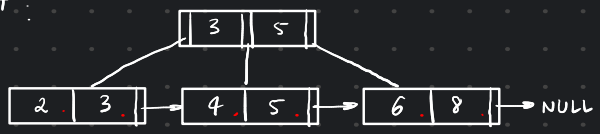
B vs B^+ trees

B :



The keys in the internal nodes are not present in the leaves.
Record pointers are present in all the nodes.

B^+ :



All the keys are present in leaves.
Data pointers/ record pointers are present only at the leaves.
Every leaf has a next node pointer. So these are more suitable for ranged queries.

Q. Insert 60, 40, 20, 50, 30 into a B^+ tree of order 5.

→ order = 5 (max # of block pointers per node)

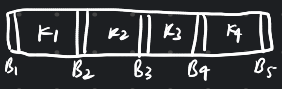
∴ max # of keys in a node = 4.



for a leaf,

4 record pointers, 1 block pointer (next)

of keys = 4



for a non-leaf,

of block pointers = 5

of keys = 4

of record pointers = 0

Insertions :

• → record/data pointer [] → block pointer

60



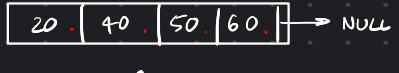
40



20



50

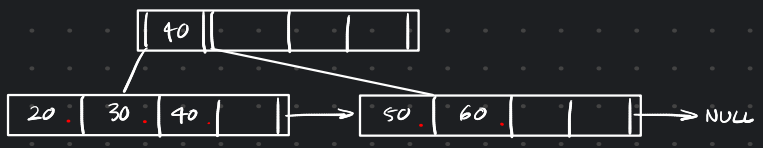


30

[20 30 40 50 60]



This is how a leaf is split.
Non-leaf nodes are split the same as in B-trees.



Insert 80.



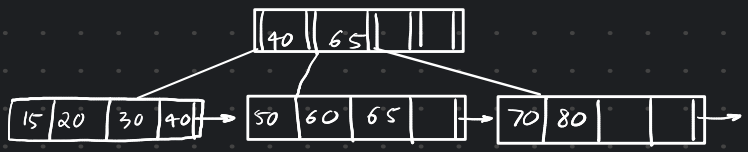
Insert 70.



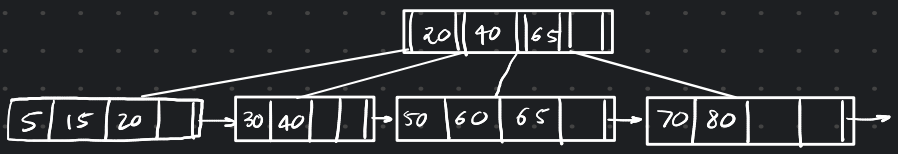
Insert 65.



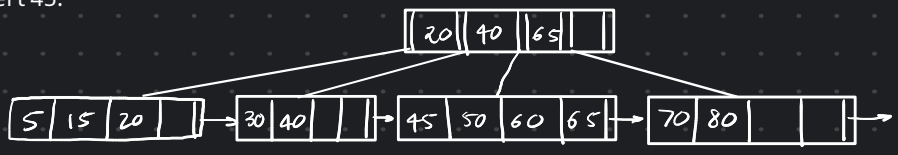
Insert 15.



Insert 5.



Insert 45.



Insert 55.



Insert 75.



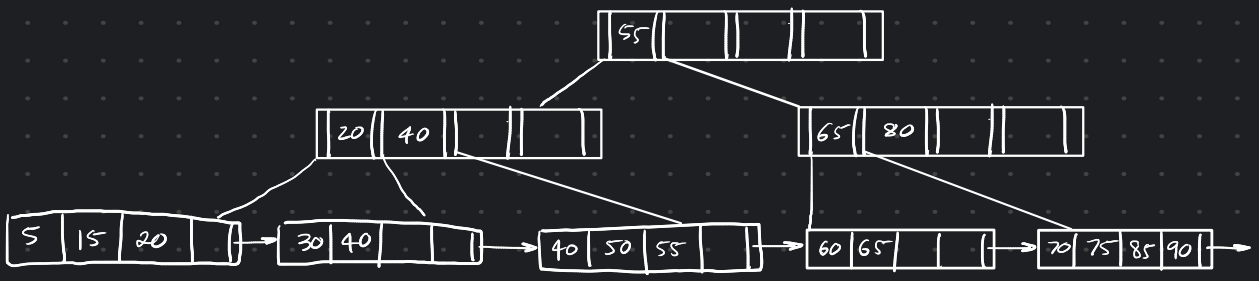
Insert 85.



Insert 90.

[20 40 55 65 80]

[70 75 80 85 90]



Q. If block size = 1024 B, search key = 10 B, block pointer = 8 B, record pointer = 9 B, then what is the order (max # of block pointers that can be stored in a node) of the B-tree?

→ order = max # of block pointers in a node.



$p(BP) + (p-1)(K+R) \le \text{Block size}$

$p(8) + (p-1)(10+9) \le 1024$

$27p \le 1043$

$p \le 38.6296$

$p = 38$

Q. Block size = 512 B, key = 5 B, BP = RP = 10 B, order p: max # of keys possible to store in a node; What is the best possible order for a B-tree node?

The order can be defined differently every time.

→



$(p+1)(BP) + p(key + RP) \le 512$

$(p+1)(10) + p(5+10) \le 512$

$25p \le 502$

$p \le 20.8$

$p = 20$

Q. Block size = 1024 B, BP = 8 B, RP = 9 B, search key = 10 B;
Order p: max pointers which can be stored in a B+ node;
What is the best possible order for a B+ tree node?
i. internal node ii. leaf

→ Internal node:





$p(B) + (p-1)(key) \le \text{Block size}$

$p(8) + (p-1)(10) \le 1024$

$18p \le 1034$

Leaf:



Total = x + 1 (Rp: p-1, BP: 1)



$(p-1)(K+R) + B \le \text{block size}$

$(p-1)(10+9) + 8 \le 1024$

$p \le 54.47$

$p = 54$

Q. Block size = 512 B, key = 5 B, BP = RP = 10 B, order p: max # of keys possible to store in a node;
What is the best possible order for a B-tree
i. internal node ii. leaf?

→ Internal node: 

Leaf node: 

$(BP)(p+1) + (Key)(p) \leq$ Block size

$10(p+1) + 5(p) \leq 512$

$15p \leq 502$

$p \leq 33.4666$

$p = 33$

$(Key + RP)p + 1(BP) \leq$ Block size

$(5+10)p + 10 \leq 512$

$15p \leq 502$

$p \leq 33$

The order is same for the internal and the external nodes as BP size = RP size.

B⁺ tree nodes can accommodate more # of keys than that of a B-tree.
If their block sizes are the same, then the order of B-tree node > order of B⁺ tree node.
of leaves in a B⁺ tree < # of nodes in a B-tree. So B⁺ trees have a lesser i/o cost than B-trees.
B⁺ trees are preferred for DB indexes.

How many minimum and maximum keys and nodes can be present in a B-tree/B⁺ tree with order 5 and 4 levels?

order p: from 2 to p BPs for the root node.
$[\lceil \frac{p}{2} \rceil, p]$ BPs for an internal node
$[\lceil \frac{p}{2} \rceil - 1, p-1]$ keys for a leaf

→

Level	max nodes	max BPs	max keys
1	1	5	4
2	5	5×5	4×5
3	25	5×25	4×25
4	125		4×125
	156		624

156 ← max # of nodes in B/B⁺ tree with order 5 and 5 levels

624 ← for B trees. For B⁺ trees, it's those in the last level, i.e. 4×125 = 500

For B⁺ trees, these are the ones that are not repeated in the upper levels. We don't discuss the rest.



In B⁺ trees, all keys are present in the leaves.
The same are repeated in the upper layers.

Level	min nodes	min BPs	min keys
1	1	2	1
2	2	$\lceil \frac{5}{2} \rceil$ → 3 × 2	3−1 → 2×2
3	6	3×6	2×6
4	18	—	$2 \times 18^{(p-1)}$
	27		53

27 ← min # of nodes in B/B⁺ tree order = 5, # of levels = 4

53 ← min # of keys

of keys in B-tree = 53
of keys in B⁺ tree = 36

How many minimum and maximum keys and nodes can be present in a B-tree/B+ tree with order 5 and 4 levels?

Order p: from 1 to 2p keys for the root.
from p to 2p keys for the rest.

→

	min keys	max keys	min BPs	max BPs
Root:	1	10	2	11
other nodes:	5	10	6	11

Level	max nodes	max BPs	max keys
1	1	11	10
2	11	11×11	10×11
3	11^2	11×11^2	10×11^2
4	11^3	11×11^3	10×11^3

max # of nodes in B/B+ tree

max # of keys in B+ tree

max # of keys in B tree

Level	min nodes	min BPs	min keys
1	1	2	1
2	2	6×2	5×2
3	12	6×12	5×12
4	72	–	5×72

min # of trees for B, B+ tree

min # of keys in a B+ tree

min # of keys in a B tree

Q. # of keys used for B+ tree indexing is 20,000. Order: p=20 (max # of pointers per node).
How many minimum levels and minimum nodes are required?

→ Say # of levels = n.

At level 1: (20 − 1) = 19 keys, 20 BPs] 1 node = 19 keys

level 2: 19 keys, 20 BPs × 20 nodes = 19×20 keys

∴ Total # of keys = $19 + 19 \times 20 + 19 \times 20^2 + \cdots + 19 \times 20^{n-1}$

$$19\left(1 + 20 + 20^2 + \cdots + 20^{n-1}\right) = 20{,}000$$

$$19 \cdot 1 \frac{(1-20^n)}{(1-20)} = 20{,}000 \Rightarrow 20^n - 1 = 20{,}000$$

$$20^n = 20{,}000 - 1$$

At last level, there'll be $\lceil \frac{20000}{19} \rceil = 1053$ pointers

$\lceil \frac{3}{20} \rceil = 1$

$\lceil \frac{53}{20} \rceil = 3$

$\lceil \frac{1053}{20} \rceil = 53$ → pointers



$\lceil \frac{20000}{19} \rceil = 1053$ → nodes in the upper level

(20,000 keys are kept in nodes